

LAPORAN PRAKTIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 7”

Disusun Oleh:

Haykal Muhammad Dahnil

2511532014

Dosen Pengampu :	Dr. Wahyudi, S.T, M.T.
Asisten Praktikum :	Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan praktikum Praktikum Struktur Data untuk pertemuan pekan ke-7 ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk mendokumentasikan hasil praktikum mengenai algoritma pengurutan dasar (Sorting). Fokus utama pada pekan ini adalah memahami logika perbandingan elemen, pertukaran data (swapping), serta visualisasi proses pengurutan data menggunakan metode Bubble Sort, Selection Sort, dan Insertion Sort baik dalam format Console maupun Graphical User Interface (GUI).

Penulis mengucapkan terima kasih kepada dosen pengampu serta asisten praktikum atas bimbingan yang telah diberikan. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan.

Padang, 21 Mei, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	3
2.1 Class BubleSort_2511532014	3
2.2 Class SelectionSort_2511532014.....	4
2.3 Class InsertionSort_2511532014.....	6
2.4 Class InsertionSortGUI_2511532014.....	7
BAB III KESIMPULAN	13
DAFTAR PUSTAKA	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data atau Sorting merupakan salah satu operasi paling mendasar dan penting dalam bidang ilmu komputer dan pengolahan data. Proses ini bertujuan untuk menyusun kembali sekumpulan data yang acak menjadi terstruktur secara teratur, baik secara menaik (ascending) dari nilai terkecil ke terbesar, maupun menurun (descending) dari nilai terbesar ke terkecil.

Terdapat berbagai macam algoritma pengurutan dasar yang memiliki cara kerja dan karakteristik memori yang unik, seperti Bubble Sort yang berbasis pertukaran tetangga secara berulang, Selection Sort yang berfokus pada pencarian nilai ekstrem di setiap iterasi, dan Insertion Sort yang mengadopsi cara manusia mengurutkan kartu di tangan. Memahami efisiensi logika dari masing-masing algoritma ini sangat penting dalam menentukan solusi pemrograman yang optimal sesuai dengan volume data yang dihadapi.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami konsep dasar, logika perulangan, dan mekanisme pertukaran data (swapping) pada algoritma Sorting.
2. Mampu mengimplementasikan algoritma Bubble Sort, Selection Sort, dan Insertion Sort menggunakan bahasa pemrograman Java.
3. Mampu menganalisis alur eksekusi, perbandingan indeks, dan kompleksitas waktu dari ketiga algoritma tersebut.
4. Mampu merancang antarmuka berbasis GUI untuk memvisualisasikan langkah demi langkah (step-by-step) jalannya proses pengurutan data secara interaktif.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini meliputi:

1. Peningkatan Kemampuan Analisis: Mahasiswa dapat membandingkan efisiensi algoritma berdasarkan jumlah operasi perulangan dan pertukaran data.

2. Penguasaan Logika Array: Mengasah keterampilan dalam memanipulasi indeks array satu dimensi melalui struktur perulangan bersarang (nested loop).
3. Pengembangan Aplikasi Interaktif: Mahasiswa memahami cara mengintegrasikan logika algoritma ke dalam komponen visual Java Swing (GUI) untuk mempermudah edukasi proses komputasi.

BAB II PEMBAHASAN

2.1 Class BubleSort_2511532014

2.1.1 Langkah Pengerjaan

1. Mendefinisikan variabel data_2014 untuk menyimpan nilai.
2. Mendeklarasikan metode 'bubbleSort_2511532014' yang menerima parameter sebuah array integer.
3. Menggunakan struktur perulangan bersarang (nested loop) 'for' untuk membandingkan dua elemen yang berdekatan.
4. Jika elemen kiri ('arr_2014[j_2014]') lebih besar daripada elemen kanan ('arr_2014[j_2014 + 1]'), lakukan proses pertukaran (swapping) menggunakan variabel bantu 'temp_2014'.
5. Dalam metode 'main', menginisialisasi array acak, mencetaknya sebelum diurutkan, memanggil metode sorting, dan mencetak kembali hasil array yang telah terurut.

2.1.2 Kode Program

```
1 package pekan7_2511532014;
2
3 public class BubleSort_2511532014 {
4     public static void bubbleSort_2511532014(int[] arr_2014) {
5         int n_2014 = arr_2014.length;
6         for (int i_2014 = 0; i_2014 < n_2014; i_2014++) {
7             for (int j_2014 = 0; j_2014 < n_2014 - 1;
8 j_2014++) {
9                 if (arr_2014[j_2014] > arr_2014[j_2014 +
10 1]) {
11                     int temp_2014 = arr_2014[j_2014];
12                     arr_2014[j_2014] =
13 arr_2014[j_2014+1];
14                     arr_2014[j_2014 + 1] = temp_2014;
15                     // System.out.println("data:"
16 arr_2014[j_2014] + " "+ arr_2014[j_2014+1]);
17                 }
18             }
19         }
20     }
21
22     public static void main(String[] args) {
23         int arr_2014[] = { 23, 78, 45, 8, 32, 56, 1 };
24         int n_2014 = arr_2014.length;
25         System.out.printf("array yang belum terurut:\n");
26         for (int i_2014 = 0; i_2014 < n_2014; i_2014++) {
27             System.out.print(arr_2014[i_2014] + " ");
28         }
29         System.out.println("");
30         bubbleSort_2511532014(arr_2014);
31         System.out.printf("array yang terurut:\n");
32         for (int i_2014 = 0; i_2014 < n_2014; i_2014++) {
33             System.out.print(arr_2014[i_2014] + " ");
34         }
35     }
36 }
```

```

30     }
31     System.out.println("");
32
33 }
34
35 }
36

```

2.1.3 Output

```

<terminated> BubleSort_2511532014 [Java Application] C:\Users\as
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78

```

2.1.4 Analisis

Algoritma Bubble Sort mengadopsi filosofi "gelembung", di mana elemen dengan nilai besar secara perlahan akan bergeser naik ke posisi akhir array di setiap iterasi luar. Analisis terhadap perulangan dalam ($j_{2014} < n_{2014} - 1$) menunjukkan bahwa algoritma ini selalu memeriksa data secara sekuensial berpasangan. Kompleksitas waktu algoritma ini adalah $O(n^2)$ dalam kondisi terburuk (worst-case) maupun rata-rata (average-case). Hal ini dikarenakan perulangan akan tetap berjalan membandingkan data meskipun array sebenarnya sudah dalam kondisi terurut di tengah iterasi.

2.2 Class SelectionSort_2511532014

2.2.1 Langkah Pengerjaan

1. Mendeklarasikan metode `selectionSort\_2511532014` dengan parameter array integer.
2. Melakukan perulangan luar dari indeks awal hingga akhir untuk menetapkan posisi target pengisian data terkecil ($minIndex_{2014} = i_{2014}$).
3. Menjalankan perulangan dalam untuk memindai sisa elemen array guna mencari nilai terkecil sesungguhnya.
4. Jika ditemukan elemen yang lebih kecil, perbarui nilai variabel `minIndex\_2014`.
5. Setelah perulangan dalam selesai, lakukan operasi pertukaran (swapping) antara elemen target `i\_2014` dengan elemen bernilai minimum yang ditemukan pada indeks `minIndex\_2014`.

2.2.2 Kode Program

```
1 package pekan7_2511532014;
2
3 public class SelectionSort_2511532014 {
4     public static void selectionSort_2511532014(int[] arr_2014) {
5         int n_2014 = arr_2014.length;
6         for (int i_2014 = 0; i_2014 < n_2014; i_2014++) {
7             int minIndex_2014 = i_2014;
8             for (int j_2014 = i_2014 + 1; j_2014 < i_2014;
9                 j_2014++) {
10                 if (arr_2014[j_2014] <
11                     arr_2014[minIndex_2014]) {
12                     minIndex_2014 = j_2014;
13                 }
14             }
15             int temp_2014 = arr_2014[i_2014];
16             arr_2014[i_2014] = arr_2014[minIndex_2014];
17             arr_2014[minIndex_2014] = temp_2014;
18         }
19     }
20
21     public static void main(String[] args) {
22         int arr_2014[] = { 23, 78, 45, 8, 32, 56, 1};
23         int n_2014 = arr_2014.length;
24         System.out.printf("array yang belum terurut:\n");
25         for (int i_2014 = 0; i_2014 < n_2014; i_2014++) {
26             System.out.print(arr_2014[i_2014] + " ");
27         }
28         System.out.println("");
29         selectionSort_2511532014(arr_2014);
30         System.out.printf("array yang terurut:\n");
31         for (int i_2014 = 0; i_2014 < n_2014; i_2014++) {
32             System.out.print(arr_2014[i_2014] + " ");
33         }
34         System.out.println("");
35     }
36 }
```

2.2.3 Output

```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.2.4 Analisis

Karakteristik utama dari Selection Sort adalah meminimalkan jumlah operasi pertukaran (swapping). Analisis baris kode menunjukkan bahwa proses penukaran data diletakkan di luar perulangan dalam ('j_2014'). Artinya, dalam satu iterasi luar,

algoritma ini hanya melakukan maksimal satu kali pertukaran data setelah menemukan elemen minimum absolut. Walaupun kompleksitas pembandingannya tetap bernilai $O(n^2)$, kinerja pemindahan datanya jauh lebih efisien dan konsisten dibandingkan dengan Bubble Sort.

2.3 Class InsertionSort_2511532014

2.3.1 Langkah Pengerjaan

1. Mendeklarasikan metode `insertionSort\_2014`. Perulangan luar dimulai dari indeks ke-1 (`i\_2014 = 1`) hingga akhir array, dan menyimpan elemen tersebut ke dalam variabel penanda (`key\_2014`).
2. Mengatur perulangan mundur (`j\_2014 = i\_2014 - 1`) untuk membandingkan elemen di sebelah kiri dengan nilai `key\_2014`.
3. Menggunakan kondisi perulangan `while (j\_2014 >= 0 && arr\_2014[j\_2014] > key\_2014)` untuk menggeser elemen yang lebih besar ke arah kanan (`arr\_2014[j\_2014 + 1] = arr\_2014[j\_2014]`).
4. Memasukkan nilai `key\_2014` ke posisi slot kosong yang tepat setelah pergeseran selesai.

2.3.2 Kode Program

```
1 package pekan7_2511532014;
2
3 public class InsertionSort_2511532014 {
4     public static void insertionSort_2014(int arr_2014[]) {
5         int n_2014 = arr_2014.length;
6         for (int i_2014 = 1; i_2014 < n_2014; i_2014++) {
7             int key_2014 = arr_2014[i_2014];
8             int j_2014 = i_2014 - 1;
9             while (j_2014 >= 0 && arr_2014[j_2014] > key_2014)
10 {
11                 arr_2014[j_2014 + 1] = arr_2014[j_2014];
12                 j_2014--;
13             }
14             arr_2014[j_2014 + 1] = key_2014;
15         }
16     }
17     public static void main(String[] args) {
18         int arr_2014[] = {23, 78, 45, 8, 32, 56, 1 };
19         int n_2014 = arr_2014.length;
20         System.out.printf("array yang belum terurut:\n");
21         for (int i_2014 = 0; i_2014 < n_2014; i_2014++) {
22             System.out.print(arr_2014[i_2014] + " ");
23         }
24         System.out.println("");
25         insertionSort_2014(arr_2014);
26         System.out.printf("array yang terurut:\n");
27         for (int i_2014 = 0; i_2014 < n_2014; i_2014++) {
28             System.out.print(arr_2014[i_2014] + " ");
29         }
30         System.out.println("");
31     }
32 }
```

```
30     }  
31  
32 }  
33
```

2.3.3 Output

```
<terminated> InsertionSort_2511532014 [Java Application] C:\Use  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

2.3.4 Analisis

Insertion Sort bekerja dengan cara membagi array secara virtual menjadi bagian yang sudah terurut (sisi kiri) dan bagian yang belum terurut (sisi kanan). Analisis pada penggunaan perulangan `while` menunjukkan aspek adaptif dari algoritma ini. Jika elemen `key\_2014` ternyata sudah berada di posisi yang benar terhadap elemen kirinya, perulangan `while` tidak akan pernah dieksekusi ($O(1)$ untuk langkah tersebut). Oleh karena itu, dalam kondisi data yang hampir terurut (nearly sorted), Insertion Sort memiliki efisiensi waktu terbaik mendekati nilai linear $O(n)$, menjadikannya algoritma pengurutan dasar paling adaptif di antara ketiganya.

2.4 Class InsertionSortGUI_2511532014

2.4.1 Langkah Pengerjaan

1. Merancang kelas `InsertionSortGUI\_2511532014` yang mewarisi (extends) `JFrame` untuk membuat jendela aplikasi Windows.
2. Menyusun tata letak (layout) menggunakan `BorderLayout` dan `FlowLayout` guna menampung input array, panel visualisasi data berbentuk `JLabel[]`, tombol eksekusi (`setButton\_2014`, `stepButton\_2014`), serta komponen `JTextArea` untuk log proses.
3. Mengimplementasikan interaksi tombol set agar menerima pola teks angka, memisahkannya dengan koma, dan menginisialisasi array visual.
4. Memodifikasi alur logika Insertion Sort konvensional ke dalam bentuk step-by-step execution di dalam metode `nextStep\_2014()`. Metode ini dieksekusi satu iterasi setiap kali tombol "Step" diklik oleh pengguna.

5. Memanggil metode `UpdateLabels\_2014()` untuk merender ulang teks angka pada label visual serta memperbarui teks laporan pada log area secara waktu nyata.

2.4.2 Kode Program

```
1 package pekan7_2511532014;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.SwingUtilities;
20
21 public class InsertionSortGUI_2511532014 extends JFrame {
22
23     private static final long serialVersionUID = 1L;
24     private int[] array_2014;
25     private JLabel[] labelArray_2014;
26     private JButton stepButton_2014, resetButton_2014,
27     setButton_2014;
28     private JTextField inputField_2014;
29     private JPanel panelArray_2014;
30     private JTextArea stepArea_2014;
31
32     private int i_2014 = 1, j_2014;
33     private boolean sorting_2014 = false;
34     private int stepCount_2014 = 1;
35
36     /**
37      * Create the frame.
38      */
39     public InsertionSortGUI_2511532014() {
40         setTitle("Insertin sort Langkah per Langkah");
41         setSize(750, 400);
42         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
43         setLocationRelativeTo(null);
44         setLayout(new BorderLayout());
45
46         //panel input
47         JPanel inputPanel_2014 = new JPanel(new FlowLayout());
48         inputField_2014 = new JTextField(30);
49         setButton_2014 = new JButton("set Array");
50         inputPanel_2014.add(new JLabel ("Masukan angka (pisahkan
51         dengan koma)"));
52         inputPanel_2014.add(inputField_2014);
```

```

51         inputPanel_2014.add(setButton_2014);
52
53         //panel array visual
54         panelArray_2014 = new JPanel();
55         panelArray_2014.setLayout(new FlowLayout());
56
57         // panel kontrol
58         JPanel controlPanel_2014 = new JPanel();
59         stepButton_2014 = new JButton("Langkah selanjutnya");
60         resetButton_2014 = new JButton("Reset");
61         controlPanel_2014.add(stepButton_2014);
62         controlPanel_2014.add(resetButton_2014);
63
64         // area text untuk log langkah langkah
65         stepArea_2014 = new JTextArea(8, 60);
66         stepArea_2014.setEditable(false);
67         stepArea_2014.setFont(new Font("Monospaced", Font.PLAIN,
14));
68         JScrollPane scrollPane_2014 = new
JScrollPane(stepArea_2014);
69
70         // tambahkan panel ke frame
71         add(inputPanel_2014, BorderLayout.NORTH);
72         add(panelArray_2014, BorderLayout.CENTER);
73         add(controlPanel_2014, BorderLayout.SOUTH);
74         add(scrollPane_2014, BorderLayout.EAST);
75
76         // event set array
77         setButton_2014.addActionListener(e ->
setArrayFromInput_2014());
78
79         // event selanjutnya
80         stepButton_2014.addActionListener(e ->
performStep_2014());
81
82         // event reset
83         resetButton_2014.addActionListener(e -> reset_2014());
84     }
85
86     private void setArrayFromInput_2014() {
87         String text_2014 = inputField_2014.getText().trim();
88         if (text_2014.isEmpty()) return;
89         String[] parts_2014 = text_2014.split(",");
90         array_2014 = new int[parts_2014.length];
91         try {
92             for (int k_2014 = 0; k_2014 < parts_2014.length;
k_2014++) {
93                 array_2014[k_2014] =
Integer.parseInt(parts_2014[k_2014].trim());
94             }
95         } catch (NumberFormatException e) {
96             JOptionPane.showMessageDialog(this, "Masukkan
hanya angka yang dipisahkan "
97                 + "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
98             return;
99         }

```

```

100         i_2014 = 1;
101         sorting_2014 = true;
102         stepButton_2014.setEnabled(true);
103         stepArea_2014.setText("");
104         panelArray_2014.removeAll();
105         labelArray_2014 = new JLabel[array_2014.length];
106         for (int k_2014 = 0; k_2014 < array_2014.length;
107             k_2014++) {
108             labelArray_2014[k_2014] = new
109                 JLabel(String.valueOf(array_2014[k_2014]));
110             labelArray_2014[k_2014].setFont(new Font("Arial",
111                 Font.BOLD, 24));
112             labelArray_2014[k_2014].setBorder(BorderFactory.createLineBorder(
113                 Color.BLACK));
114             labelArray_2014[k_2014].setPreferredSize(new
115                 Dimension(50, 50));
116             labelArray_2014[k_2014].setHorizontalAlignment(SwingConstants.C
117                 ENTER);
118             panelArray_2014.add(labelArray_2014[k_2014]);
119         }
120         panelArray_2014.revalidate();
121         panelArray_2014.repaint();
122     }
123
124     private void performStep_2014() {
125         if (i_2014 < array_2014.length && sorting_2014) {
126             int key_2014 = array_2014[i_2014];
127             j_2014 = i_2014 - 1;
128
129             StringBuilder stepLog_2014 = new StringBuilder();
130             stepLog_2014.append("Langkah
131 ").append(stepCount_2014).
132             append(": Memasukkan
133 ").append(key_2014).append("\n");
134
135             while(j_2014 >= 0 && array_2014[j_2014] >
136                 key_2014) {
137                 array_2014[j_2014 + 1] =
138                 array_2014[j_2014];
139                 j_2014--;
140             }
141             array_2014[j_2014 + 1] = key_2014;
142
143             UpdateLabels_2014();
144             stepLog_2014.append("Hasil:
145 ").append(arrayToString(array_2014)).append("\n\n");
146             stepArea_2014.append(stepLog_2014.toString());
147
148             i_2014++;
149             stepCount_2014++;
150
151             if (i_2014 == array_2014.length) {
152                 sorting_2014 = false;
153                 stepButton_2014.setEnabled(false);

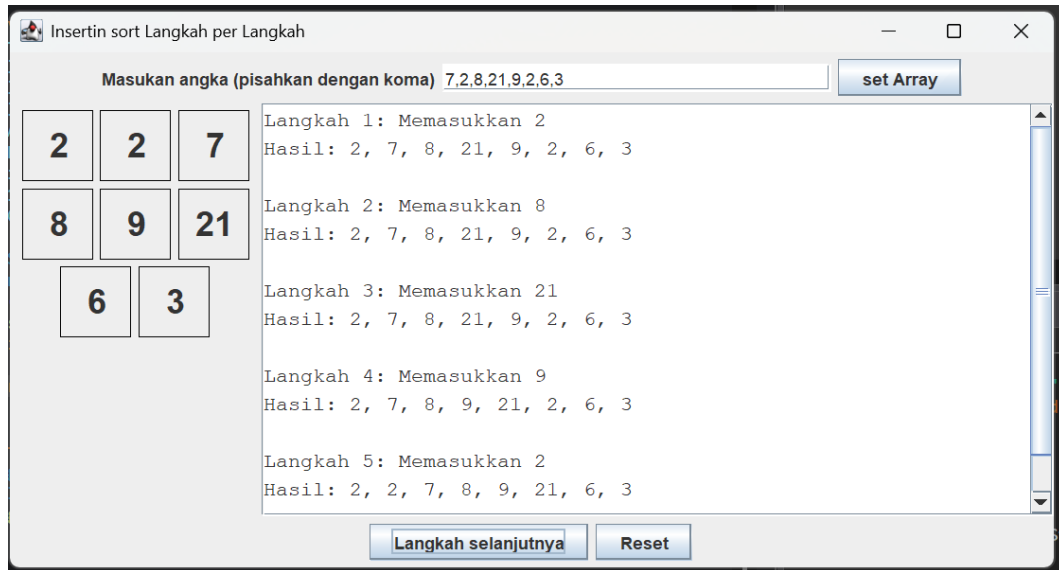
```

```

143         JOptionPane.showMessageDialog(this,
"Sorting selesai!");
144     }
145 }
146 }
147
148 private void UpdateLabels_2014() {
149     for (int k_2014 = 0; k_2014 < array_2014.length;
k_2014++) {
150         labelArray_2014[k_2014].setText(String.valueOf(array_2014[k_2014]));
151     }
152 }
153
154 private void reset_2014() {
155     inputField_2014.setText("");
156     panelArray_2014.removeAll();
157     panelArray_2014.revalidate();
158     panelArray_2014.repaint();
159     stepArea_2014.setText("");
160     stepButton_2014.setEnabled(false);
161     sorting_2014 = false;
162     i_2014 = 1;
163     stepCount_2014 = 1;
164 }
165 private String arrayToString(int[] arr_2014) {
166     StringBuilder sb_2014 = new StringBuilder();
167     for (int k_2014 = 0; k_2014 < arr_2014.length; k_2014++)
168     {
169         sb_2014.append(arr_2014[k_2014]);
170         if (k_2014 < arr_2014.length - 1)
171             sb_2014.append(", ");
172     }
173     return sb_2014.toString();
174 }
175
176 public static void main(String[] args) {
177     SwingUtilities.invokeLater(() -> {
178         InsertionSortGUI_2511532014 gui = new
InsertionSortGUI_2511532014();
179         gui.setVisible(true);
180     });
181 }
182

```

2.4.3 Output



2.4.4 Analisis

Pembuatan aplikasi berbasis GUI ini merubah paradigma eksekusi algoritma dari yang semula bersifat otomatis linier di memori komputer menjadi objek interaktif yang dikendalikan oleh event-driven programming (aksi klik pengguna). Variabel iterasi `i` diubah dari variabel lokal perulangan menjadi atribut kelas (instance variable) agar status penanda indeks pengurutan tidak hilang di antara klik tombol "Step" yang terpisah. Modifikasi ini sangat sukses dalam memberikan sarana bantu ajar visual untuk melacak pergerakan elemen array dan melihat bagaimana sub-array bagian kiri berangsur-angsur menjadi terurut stabil.

BAB III

KESIMPULAN

Berdasarkan praktikum pekan 6, dapat disimpulkan bahwa:

1. Mekanisme Kerja Berbeda: Bubble Sort mengandalkan pertukaran nilai konsekutif, Selection Sort mengisolasi nilai minimum terlebih dahulu sebelum melakukan penukaran tunggal, dan Insertion Sort bekerja secara dinamis menggeser elemen untuk menyisipkan nilai penanda di tempat yang tepat.
2. Efisiensi dan Adaptivitas: Meskipun ketiga algoritma dasar memiliki kompleksitas waktu teoretis rata-rata sebesar $O(n^2)$, Insertion Sort terbukti memiliki kinerja terbaik dalam kondisi nyata karena sifat perulangannya yang adaptif terhadap data yang sudah terurut sebagian.
3. Pentingnya Visualisasi Interaktif: Melalui implementasi komponen Java Swing ('JFrame'), alur kalkulasi internal algoritma dapat dipetakan secara grafis melalui log teks terstruktur, membantu programmer memahami mutasi data array di setiap tahapan eksekusi program.

DAFTAR PUSTAKA

Laporan Tugas

1. Class Mahasiswa_2511532014

a. Langkah Pengerjaan

1. Membuat kelas model bernama Mahasiswa_2511532014 di dalam package pekan7_2511532014.
2. Mendeklarasikan tiga atribut data dengan hak akses terenkapsulasi (private String nama_2014, nim_2014, prodi_2014).
3. Membuat sebuah konstruktor bertipe publik untuk menginisialisasi nilai ketiga atribut tersebut secara langsung saat objek instansiasi diciptakan di memori.
4. Menyediakan rangkaian metode Setter (setNama_2014, setNim_2014, setProdi_2014) untuk memberikan otorisasi perubahan nilai data dari luar kelas secara aman.
5. Menyediakan rangkaian metode Getter (getNama_2014, getNim_2014, getProdi_2014) untuk mengembalikan isi nilai data dari masing-masing atribut terenkapsulasi tersebut saat program utama membutuhkannya.

b. Kode Program

```
1. package pekan7_2511532014;
2.
3. public class Mahasiswa_2511532014 {
4.     private String nama_2014;
5.     private String nim_2014;
6.     private String prodi_2014;
7.
8.     // konstruktor
9.     public Mahasiswa_2511532014(String nama_2014, String
nim_2014, String prodi_2014) {
10.         this.nama_2014 = nama_2014;
11.         this.nim_2014 = nim_2014;
12.         this.prodi_2014 = prodi_2014;
13.     }
14.
15.     //setter
16.     public void setNama_2014(String nama_2014){
17.         this.nama_2014 = nama_2014;
18.     }
19.     public void setNim_2014(String nim_2014) {
20.         this.nim_2014 = nim_2014;
21.     }
22.     public void setProdi_2014(String prodi_2014) {
23.         this.prodi_2014 = prodi_2014;
24.     }
25.
26.     //getter
27.     public String getNama_2014() {
28.         return nama_2014;
29.     }
30.     public String getNim_2014() {
31.         return nim_2014;
```

```

32.     }
33.     public String getProdi_2014() {
34.         return prodi_2014;
35.     }
36.
37.     @Override
38.     public String toString() {
39.         return nama_2014;
40.     }
41.
42. }
43.

```

c. Analisis

Kelas Mahasiswa_2511532014 mengadopsi pola perancangan Data Transfer Object (DTO) atau kelas entitas murni. Analisis pada arsitektur kelas ini sangat krusial karena ia membungkus tipe data string majemuk menjadi satu kesatuan unit objek struktural tunggal. Penerapan kata kunci `private` pada variabel objek menjamin proteksi data dari manipulasi langsung yang tidak tervalidasi. Penggunaan metode Getter—khususnya `getNama_2014()`—menjadi gerbang interaksi utama yang akan dimanfaatkan oleh kelas eksekutor antarmuka (GUI) untuk membandingkan rangkaian string nama antar mahasiswa secara leksikografis dalam proses pengurutan.

2. Class Musik_2511532014

a. Langkah Pengerjaan

1. Merancang kelas jendela utama grafis `sortingGUI_2511532014` yang mewarisi komponen `JFrame`.
2. Mendeklarasikan `ArrayList<Mahasiswa_2511532014>` untuk menampung daftar data dinamis mahasiswa di dalam memori program.
3. Menyusun komponen input data teks (`txtNama_2014`, `txtNim_2014`, `txtProdi_2014`) menggunakan tata letak `GridLayout` pada panel kiri atas, serta membuat komponen drop-down pilihan `JComboBox` untuk menentukan pilihan algoritma sorting yang diinginkan pengguna (Bubble Sort, Selection Sort, atau Insertion Sort).
4. Membuat objek komponen `JTable` dan diikatkan dengan `DefaultTableModel` untuk menampilkan visualisasi basis data tabel mahasiswa secara terstruktur di layar komputer.
5. Membangun area teks besar kanan (`areaProses_2014`) yang dilapisi dengan `JScrollPane` untuk mencetak secara detail log pergerakan string nama pada setiap siklus perulangan luar (pass-by-pass).
6. Mengimplementasikan metode sorting berbasis perbandingan teks string dengan memanfaatkan instruksi `compareToIgnoreCase()`.

Struktur logika sorting konvensional dimodifikasi agar setiap kali satu putaran outer loop selesai, sistem akan merender ulang urutan nama mahasiswa terkini ke dalam areaProses_2014.

7. Mengikat tombol tambah (btnTambah_2014) dengan fungsi pembaca isi kolom teks, memasukkannya ke dalam ArrayList, dan memperbarui visual baris tabel lewat metode addRow().
8. Mengikat tombol pengurutan (btnSorting_2014) dengan struktur kontrol kondisional switch-case untuk mengeksekusi metode algoritma sorting yang bersesuaian dengan item pilihan drop-down terpilih, lalu memperbarui susunan baris pada tabel utama di akhir proses.

b. Kode Program

```
1. package pekan7_2511532014;
2.
3. import java.awt.BorderLayout;
4. import java.awt.Color;
5. import java.awt.Dimension;
6. import java.awt.Font;
7. import java.awt.GridLayout;
8. import java.util.ArrayList;
9.
10. import javax.swing.BorderFactory;
11. import javax.swing.JButton;
12. import javax.swing.JComboBox;
13. import javax.swing.JFrame;
14. import javax.swing.JLabel;
15. import javax.swing.JOptionPane;
16. import javax.swing.JPanel;
17. import javax.swing.JScrollPane;
18. import javax.swing.JSplitPane;
19. import javax.swing.JTable;
20. import javax.swing.JTextArea;
21. import javax.swing.JTextField;
22. import javax.swing.SwingConstants;
23. import javax.swing.table.DefaultTableModel;
24.
25. public class sortingGUI_2511532014 extends JFrame {
26.
27.     private static final long serialVersionUID = 1L;
28.     private JTextField txtNama_2014;
29.     private JTextField txtNim_2014;
30.     private JTextField txtProdi_2014;
31.
32.     private JButton btnTambah_2014;
33.     private JButton btnHapus_2014;
34.     private JButton btnSorting_2014;
35.
36.     private JComboBox<String> comboSorting_2014;
37.
38.     private JTable table_2014;
39.     private DefaultTableModel model_2014;
40.
41.     private JTextArea areaProses_2014;
```

```

42.
43.     private ArrayList<Mahasiswa_2511532014>
        listMahasiswa_2014;
44.
45.     /**
46.      * Create the frame.
47.      */
48.     public sortingGUI_2511532014() {
49.         setTitle("Sorting Mahasiswa - 2511532014");
50.         setSize(900, 600);
51.         setLocationRelativeTo(null);
52.         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
53.         setLayout(new BorderLayout());
54.
55.         listMahasiswa_2014 = new ArrayList<>();
56.
57.         // judul
58.         JLabel lblJudul_2014 = new JLabel("PROGRAM SORTING
        DATA MAHASISWA", SwingConstants.CENTER);
59.
60.         lblJudul_2014.setFont(new Font("Arial", Font.BOLD,
        22));
61.         lblJudul_2014.setOpaque(true);
62.         lblJudul_2014.setBackground(new Color(30, 144,
        255));
63.         lblJudul_2014.setForeground(Color.WHITE);
64.         lblJudul_2014.setPreferredSize(new Dimension(100,
        50));
65.
66.         add(lblJudul_2014, BorderLayout.NORTH);
67.
68.         //Panel input
69.         JPanel panelInput_2014 = new JPanel(new
        GridLayout(5, 2, 10, 10));
70.
71.         panelInput_2014.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));
72.
73.         panelInput_2014.add(new JLabel("Nama Mahasiswa"));
74.         txtNama_2014 = new JTextField();
75.         panelInput_2014.add(txtNama_2014);
76.
77.         panelInput_2014.add(new JLabel("NIM"));
78.         txtNim_2014 = new JTextField();
79.         panelInput_2014.add(txtNim_2014);
80.
81.         panelInput_2014.add(new JLabel("Program Studi"));
82.         txtProdi_2014 = new JTextField();
83.         panelInput_2014.add(txtProdi_2014);
84.
85.         panelInput_2014.add(new JLabel("Pilih Sorting"));
86.         comboSorting_2014 = new JComboBox<>();
87.         comboSorting_2014.addItem("Insertion Sort");
88.         comboSorting_2014.addItem("Selection Sort");
89.         comboSorting_2014.addItem("Bubble Sort");
90.
91.         panelInput_2014.add(comboSorting_2014);

```

```

91.
92.         btnTambah_2014 = new JButton("Tambah Data");
93.         btnHapus_2014 = new JButton("Hapus Data");
94.         btnSorting_2014 = new JButton("Mulai Sorting");
95.
96.         panelInput_2014.add(btnTambah_2014);
97.         panelInput_2014.add(btnHapus_2014);
98.
99.         add(panelInput_2014, BorderLayout.WEST);
100.
101.         // Tabel
102.         model_2014 = new DefaultTableModel();
103.         model_2014.addColumn("Nama");
104.         model_2014.addColumn("NIM");
105.         model_2014.addColumn("Program Studi");
106.
107.         table_2014 = new JTable(model_2014);
108.
109.         // Tabel
110.         JScrollPane scrollTable_2014 = new
JScrollPane(table_2014);
111.
112.         scrollTable_2014.setBorder(BorderFactory.createTitledBord
er("Data Mahasiswa"));
113.
114.         // Area Proses ← INI YANG HILANG,
115.         // tambahkan di sini
116.         areaProses_2014 = new JTextArea();
117.         areaProses_2014.setEditable(false);
118.         areaProses_2014.setFont(new
Font("Monospaced", Font.PLAIN, 13));
119.
120.         JScrollPane scrollArea_2014 = new
JScrollPane(areaProses_2014);
121.
122.         scrollArea_2014.setBorder(BorderFactory.createTitledBorde
r("Proses Sorting"));
123.
124.         // Panel Proses
125.         JPanel panelProses_2014 = new JPanel(new
BorderLayout(5, 5));
126.         panelProses_2014.add(btnSorting_2014,
BorderLayout.NORTH);
127.         panelProses_2014.add(scrollArea_2014,
BorderLayout.CENTER); // ← bukan scrollTable!
128.
129.         // JSplitPane
130.         JSplitPane splitPane_2014 = new
JSplitPane(JSplitPane.VERTICAL_SPLIT, scrollTable_2014,
panelProses_2014);
131.         splitPane_2014.setDividerLocation(280);
132.         splitPane_2014.setResizeWeight(0.55);
133.         splitPane_2014.setOneTouchExpandable(true);
134.         add(splitPane_2014, BorderLayout.CENTER);
135.
136.         // Action Button

```

```

134.         btnTambah_2014.addActionListener(e ->
        tambahData_2014());
135.         btnHapus_2014.addActionListener(e ->
        hapusData_2014());
136.         btnSorting_2014.addActionListener(e ->
        mulaiSorting_2014());
137.
138.     }
139.
140.     // method tambah data
141.     private void tambahData_2014() {
142.         String nama_2014 = txtNama_2014.getText();
143.         String nim_2014 = txtNim_2014.getText();
144.         String prodi_2014 =
        txtProdi_2014.getText();
145.         if (nama_2014.isEmpty() ||
        nim_2014.isEmpty() || prodi_2014.isEmpty()) {
146.             JOptionPane.showMessageDialog(this,"Semua data wajib
        diisi!");
147.             return;
148.         }
149.         Mahasiswa_2511532014 mhs_2014 = new
        Mahasiswa_2511532014(nama_2014, nim_2014, prodi_2014);
150.         listMahasiswa_2014.add(mhs_2014);
151.         model_2014.addRow(new Object[]{nama_2014,
        nim_2014, prodi_2014});
152.
153.         txtNama_2014.setText("");
154.         txtNim_2014.setText("");
155.         txtProdi_2014.setText("");
156.     }
157.
158.     // method hapus data
159.     private void hapusData_2014() {
160.         int baris_2014 =
        table_2014.getSelectedRow();
161.         if (baris_2014 == -1) {
162.             JOptionPane.showMessageDialog(this,
        "Pilih data yang ingin dihapus!");
163.             return;
164.         }
165.
166.         listMahasiswa_2014.remove(baris_2014);
167.         model_2014.removeRow(baris_2014);
168.         JOptionPane.showMessageDialog(this, "Data
        berhasil dihapus!");
169.     }
170.
171.     //method sorting
172.     private void mulaiSorting_2014() {
173.         areaProses_2014.setText("");
174.         String pilihan_2014 =
        comboSorting_2014.getSelectedItem().toString();
175.         ArrayList<Mahasiswa_2511532014> temp_2014 =
        new ArrayList<>(listMahasiswa_2014);

```

```

176.         if (pilihan_2014.equals("Insertion Sort"))
177.         {
178.             insertionSort_2014(temp_2014);
179.         } else if (pilihan_2014.equals("Selection
Sort")) {
180.             selectionSort_2014(temp_2014);
181.         } else {
182.             bubbleSort_2014(temp_2014);
183.         }
184.         tampilkanTabel_2014(temp_2014);
185.
186.         //Insertion Sort
187.         private void
insertionSort_2014(ArrayList<Mahasiswa_2511532014> list_2014) {
188.             areaProses_2014.append("== INSERTION SORT
==\n\n");
189.             for (int i_2014 = 1; i_2014 <
list_2014.size(); i_2014++) {
190.                 Mahasiswa_2511532014 key_2014 =
list_2014.get(i_2014);
191.                 int j_2014 = i_2014 - 1;
192.                 while (j_2014 >= 0 &&
list_2014.get(j_2014).getNama_2014().compareToIgnoreCase(key_20
14.getNama_2014()) > 0) {
193.                     list_2014.set(j_2014 +
1, list_2014.get(j_2014));
194.                     j_2014--;
195.                 }
196.                 list_2014.set(j_2014 + 1, key_2014);
197.                 areaProses_2014.append("Langkah " +
i_2014 + " : "+ tampilNama_2014(list_2014) + "\n");
198.             }
199.         }
200.
201.         // SelectionSort
202.         private void
selectionSort_2014(ArrayList<Mahasiswa_2511532014> list_2014) {
203.             areaProses_2014.append("=== SELECTION SORT
===\n\n");
204.             for (int i_2014 = 0; i_2014 <
list_2014.size() - 1; i_2014++) {
205.                 int min_2014 = i_2014;
206.                 for (int j_2014 = i_2014 + 1; j_2014
< list_2014.size(); j_2014++) {
207.                     if
(list_2014.get(j_2014).getNama_2014().compareToIgnoreCase(list_
2014.get(min_2014).getNama_2014()) < 0) {
208.                         min_2014 = j_2014;
209.                     }
210.                 }
211.                 Mahasiswa_2511532014 temp_2014 =
list_2014.get(min_2014);
212.                 list_2014.set(min_2014,
list_2014.get(i_2014));
213.                 list_2014.set(i_2014, temp_2014);

```



```

214.         areaProses_2014.append("Pass " +
215.             (i_2014 + 1) + " : " + tampilNama_2014(list_2014) + "\n");
216.     }
217.
218.     // BubbleSort
219.     private void
220.         bubbleSort_2014(ArrayList<Mahasiswa_2511532014> list_2014) {
221.         areaProses_2014.append("=== BUBBLE SORT
222.             ===\n\n");
223.         for (int i_2014 = 0; i_2014 < list_2014.size() -
224.             1; i_2014++) {
225.             for (int j_2014 = 0; j_2014 <
226.                 list_2014.size() - i_2014 - 1; j_2014++) {
227.                 if
228.                     (list_2014.get(j_2014).getNama_2014().compareToIgnoreCase(list_
229.                         2014.get(j_2014 + 1).getNama_2014())) > 0) {
230.                     Mahasiswa_2511532014 temp_2014
231.                         = list_2014.get(j_2014);
232.                     list_2014.set(j_2014,
233.                         list_2014.get(j_2014 + 1));
234.                     list_2014.set(j_2014 + 1, temp_2014);
235.                 }
236.                 areaProses_2014.append("Pass " + (i_2014 +
237.                     1) + " : " + tampilNama_2014(list_2014) + "\n");
238.             }
239.         }
240.
241.     // method tampil nama
242.     private String
243.         tampilNama_2014(ArrayList<Mahasiswa_2511532014> list_2014) {
244.         String hasil_2014 = "[";
245.         for (int i_2014 = 0; i_2014 <
246.             list_2014.size(); i_2014++) {
247.             hasil_2014 +=
248.                 list_2014.get(i_2014).getNama_2014();
249.             if (i_2014 != list_2014.size() - 1) {
250.                 hasil_2014 += ", ";
251.             }
252.             hasil_2014 += "]";
253.             return hasil_2014;
254.         }
255.
256.     //update table
257.     private void
258.         tampilkanTabel_2014(ArrayList<Mahasiswa_2511532014> list_2014)
259.         {
260.             model_2014.setRowCount(0);
261.             for (Mahasiswa_2511532014 mhs_2014 :
262.                 list_2014) {
263.                 model_2014.addRow(new
264.                     Object[]{mhs_2014.getNama_2014(), mhs_2014.getNim_2014(),
265.                         mhs_2014.getProdi_2014()});
266.             }
267.         }

```

```

253.
254.         //main
255.         public static void main(String[] args) {
256.             new
257.             sortingGUI_2511532014().setVisible(true);
258.         }
259.     }
260.

```

c. Output

d. Analisis

Program aplikasi `sortingGUI_2511532014` ini berhasil menyatukan konsep manipulasi objek, penyimpanan dinamis `ArrayList`, dan logika algoritma pengurutan dasar ke dalam visualisasi antarmuka berbasis GUI yang kompleks. Analisis mendalam pada fungsionalitas perbandingan data memperlihatkan pergeseran teknik komputasi: jika pada sub-bab sebelumnya (2.1 hingga 2.3) perbandingan array didasarkan pada operator matematika bilangan integer sederhana (`>`), maka pada aplikasi pengurutan mahasiswa ini perbandingan wajib menggunakan operasi fungsi `compareToIgnoreCase()`.

Fungsi bawaan kelas `String` Java ini mengembalikan nilai integer positif jika urutan karakter string objek saat ini secara leksikografis (alfabetis) bernilai lebih besar daripada string objek target di dalam parameter. Modifikasi penempatan instruksi pencetakan log di akhir perulangan luar memberikan pemahaman visual yang objektif mengenai

bagaimana sekumpulan objek record data di memori heap komputer dimutasi posisinya satu per satu di setiap tahapan lingkaran siklus operasional komputer (pass), hingga menghasilkan susunan tabel data yang rapi dan terurut stabil.